

Exercise 2: Link State and Distance Vector Routing

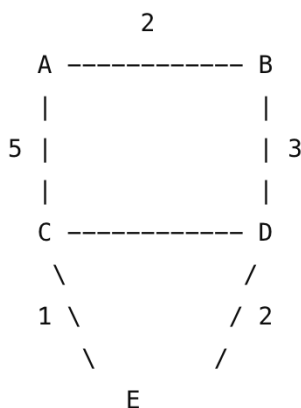
Name: นายวสุสิทธิ์ พวงขุนเทียน

Student ID: 6710451291

Date: 21 กันยายน 2569

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

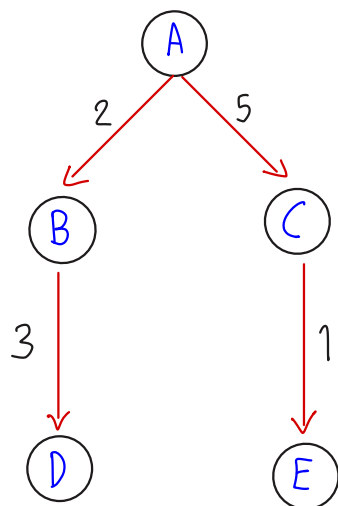
Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞	∞
1	B	2, A	5, A	5, B	∞
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



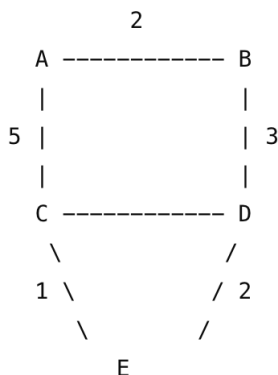
Destination	Shortest Path	Total Cost
B	A-B	2
C	A-C	5
D	A-B-D	5
E	A-C-E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A-B
C	5	C	A-C
D	5	B	A-B-D
E	6	C	A-C-E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2 + 0 = 2$	$5 + 4 = 9$	2	B
C	$2 + 4 = 6$	$5 + 0 = 5$	5	C
D	$2 + 3 = 5$	$5 + 1 = 6$	5	B
E	$2 + 5 = 7$	$5 + 1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A-B
C	C	5	A-C
D	B	5	A-B-D
E	C	6	A-C-E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- 1 • Which router(s) detect the failure.
- 2 • What information is flooded through the network.
- 3 • How routers update their link-state databases.
- 4 • How routers calculate new shortest paths.

Distance Vector routing.

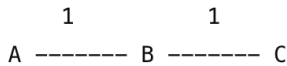
Your answer should describe:

- 1 • How routers learn about the failure.
- 2 • How updated distance vectors propagate.
- 3 • Why routers may temporarily have incorrect routing information.
- 4 • How routing eventually converges.

	Link State	Distance Vector
1	Router C และ D ที่เชื่อมต่อกับ link โดยตรง จะเห็น link failure	Router C และ D เจอ link failure และอัปเดต cost ของเส้นทางนั้นให้เป็น infinity
2	ข้อมูล LSPs (Link State Packets) แบบใหม่ที่จะบอกว่า link ทด จะถูก flood ไปยังทุก router ใน network	Router ที่เจอจะส่ง Routing update (Distance Vector) ชุดใหม่ไปให้เฉพาะ neighbor ของตนเองเท่านั้น
3	ทุก router ที่ได้รับ LSPs จะนำข้อมูลไปอัปเดต Link-State Databases ของตนเองให้เป็นภาพเดียวกัน	อาจเกิด Routing loop ชั่วคราวเนื่องจากอัปเดตกระจายแบบ Routing by rumor ทำให้บาง router ยังใช้ข้อมูลเก่าที่ล้าสมัย
4	แต่ละ router จะรัน Dijkstra's algorithm ด้วยตนเองเพื่อหา Shortest-Path Tree และสร้าง Forwarding table ใหม่	Network จะ converge เมื่อ Distance vectors ถูกอัปเดตและคำนวณผ่าน Bellman-Ford equation ไปเรื่อยๆ จนกว่าค่า cost ทั้งระบบจะนิ่ง

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?
2. Show how the estimated distance to C can increase over successive iterations.
3. What is count-to-infinity?
4. Why does it occur in Distance Vector routing?
5. Name a mitigation approach.

1. Why can this create a routing loop?

คำตอบ: เมื่อลิงก์ B–C หาด B จะพยายามหาทางไป C ใหม่ ซึ่ง B ได้รับข้อมูลเก่าจาก A ที่ A ไป C ได้ใน Cost 2 B จึงคิดว่าสามารถไป C ผ่าน A ได้ (โดยไม่รู้ว่าจริงๆ แล้ว A ก็ต้องพึ่งเส้นทางผ่าน B) ทำให้ B โยน packet ไปหา A และ A โยนกลับหา B เกิดเป็น Routing loop

2. Show how the estimated distance to C can increase over successive iterations.

- Iteration 1 : B อัปเดต $D_B(C) = c(B,A) + D_A(C) = 1 + 2 = 3$ แล้วส่งไปให้ A
- Iteration 2 : A รับข้อมูลใหม่จาก B และอัปเดต $D_A(C) = c(A,B) + D_B(C) = 1 + 3 = 4$ แล้วส่งให้ B
- Iteration 3 : B อัปเดต $D_B(C) = 1 + 4 = 5$ ค่า Cost จะบวกเพิ่มขึ้นเรื่อยๆ ไปเรื่อยๆ สลับกันไปมา

3. What is count-to-infinity?

คำตอบ: ปัญหาที่เกิดขึ้นเมื่อมี Link failure ทำให้เกิด Routing loop ภายในโปรโตคอล DV ซึ่งโหนดจะทำการบอกค่า Distance สลับกันไปมาจนกว่าจะถึงค่าที่กำหนดไว้ว่าเป็น Infinity (เช่น 16 ใน RIP) จึงหยุดดูและลบเส้นทางนั้นทิ้ง

4. Why does it occur in Distance Vector routing?

คำตอบ: เพราะ Distance Vector ทำงานแบบ Routing by rumor โหนดจะรู้เฉพาะค่า Cost รวมและ Next hop จาก neighbor แต่ไม่มีข้อมูล Global Topology จึงไม่ทราบค่าเส้นทางที่ neighbor และนั่นมันวนกลับมาหาตัวเอง หรือไม่

5. Name a mitigation approach.

คำตอบ: ใช้เทคนิค Poisoned Reverse หรือ Split Horizon (หาก A ใช้เส้นทางผ่าน B เพื่อไป C, A จะบอก B ว่าระยะทางจาก A ไป C คือ infinity เพื่อป้องกันไม่ให้ B ใช้ A เป็นทางผ่านกลับไป C)

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra's Algorithm	Bellman - Ford Algorithm
Information maintained	Link - State Database (LSDB) (Global Map)	Distance Tables/Distance Vectors
Information exchanged	Link และ Cost เหนือของตนเอง (Link State Packet (LSPs))	Routing table ที่จุดของตนเอง
Algorithm	Centralized หรือ Global	Decentralized หรือ Distributed
Topology knowledge	Global /Complete Topology	Partial (รู้แค่ neighbor และ Cost วนๆ)
Routing updates	Flooding ไปยังทุกโหนดในเครือข่าย	ส่งให้เฉพาะ router ที่เชื่อมต่อโดยตรง
Convergence	Fast	Slow
Routing loops	เกิดขึ้นได้ยากมาก	เกิดขึ้นได้ง่าย
Count-to-infinity	ไม่เกิดปัญหานี้	เกิดปัญหานี้ได้ง่ายเมื่อ Link failure
Example protocol	OSPF, IS-IS	RIP, BGP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

เลือกใช้ Link State (เช่น OSPF) โดยมีเหตุผลดังนี้

1. Fast Convergence : เมื่อเกิด Link failure ระบบจะทำการ Flooding ข้อมูล LSPs ทันที ทำให้ Router ทุกตัวอัปเดตเส้นทางได้รวดเร็ว ลดเวลา Downtime ได้ดีกว่า Distance Vector ที่ต้องรอส่งข้อมูลแบบทอดต่อกทอด

2. Accurate Topology Knowledge : ทุก Router มีข้อมูล Global map (LSDB) ของทั้งเครือข่ายที่สมบูรณ์แบบ ทำให้คำนวณเส้นทางได้แม่นยำ ป้องกันการเกิด Routing loops หรือปัญหา Count-to-infinity

3. Hierarchical Design : โปรโตคอล Link State รองรับการจัดแบบเครือข่ายแบบลำดับชั้น

(เช่น การแบ่ง OSPF Areas) ซึ่งจำเป็นอย่างยิ่งสำหรับ Enterprise ที่มี Router ถึง 500 ตัว เพราะช่วยจำกัดขอบเขตการทำ Flooding และลดขนาด Routing table ได้อย่างมีประสิทธิภาพ

ข้อเสียของ Link State : ต้องแลกมากับการใช้ทรัพยากร CPU และ Memory บน Router ที่สูงกว่า Distance Vector เพื่อใช้ในการเก็บ LSDB ขนาดใหญ่และการประมวลผล Dijkstra's algorithm

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

เปรียบเทียบการทำงานของ Link State และ Distance Vector ได้ดังนี้

1. Information maintained : LS เก็บข้อมูลทั้งเครือข่าย (LSDB) ขนาด $O(N+E)$ ส่วน DV เก็บแค่ตารางระยะทางของ router ที่เชื่อมต่อกันโดยตรง (neighbor)
2. Communication overhead : LS มีการ Flood ข้อมูลทั้งเครือข่าย เกิด Overhead $O(N \times E)$ ส่วน DV สื่อสารเฉพาะกับ router ที่เชื่อมต่อกันโดยตรง (neighbor)
3. Computation : LS ใช้ Dijkstra $O(N \log N)$ ซึ่งกินทรัพยากรสูงกว่าสมการ Bellman-Ford ของ DV
4. Convergence : LS ค้นหาเส้นทางที่ถูกต้องได้เร็ว ส่วน DV ช้าและเสี่ยงต่อการเกิด Count-to-infinity
5. Scalability : LS รองรับการจัดแบบลำดับชั้น (Hierarchical) ทำให้ขยายขนาดได้ดีกว่า DV

เหตุผลที่เครือข่ายขนาดใหญ่นิยมใช้ Link State เพราะ LS ป้องกันปัญหา Routing loops ได้ดีได้จากการเห็นภาพรวม (Complete Topology) และปรับตัวได้เร็วเมื่อ Link failure (Fast convergence) นอกจากนี้ยังรองรับการแบ่ง Area เพื่อจำกัดปริมาณข้อมูล (Hierarchical routing) ทำให้มีประสิทธิภาพสูงคุ้มค่ากับทรัพยากร router ที่ต้องใช้เพิ่มขึ้น